

B2 – INTEGRATION

Campus Companion —Formation B3

Table des matières

1. Objet de l'annexe	1
2. Panorama des trois intégrations	1
3. Intégration mobile (Flutter / Dart)	2
3.1 Configuration (ollama_config.dart)	2
3.2 Appel du chat (ollama_client.dart)	2
3.3 Fallback ticket (ticket_client.dart)	3
4. Intégration web (React)	3
4.1 Extrait — src/services/api.js	3
5. Intégration desktop (Electron)	4
5.1 Extrait — renderer/app.js	4
6. Ce que les trois intégrations ont en commun	5
7. Lien avec le référentiel de compétence	Erreur ! Signet non défini.

1. Objet de l'annexe

Cette annexe complète l'Annexe B1 (Documentation API) et apporte une preuve opérationnelle à la compétence C1.3 — « Programmation de briques et services logiciels. Conception de services métiers » : la même API backend Campus Companion a été intégrée, de façon indépendante, par trois clients distincts (mobile Flutter, web React, desktop Electron), chacun développé par un membre différent de l'équipe.

Le référentiel exige de produire du logiciel générique réutilisable. La preuve la plus solide de cette généricité n'est pas déclarative : elle réside dans le code d'intégration réellement écrit par chaque client, disponible dans le dépôt github.com/julien-gournay/ycompagnon-apps. Cette annexe présente, pour chacun des trois clients, la couche d'intégration qu'il a mise en place pour consommer l'API, et met en évidence ce qui reste identique d'un client à l'autre.

2. Panorama des trois intégrations

Client	Technologie	Fichiers d'intégration	Configuration de l'URL
Mobile	Flutter / Dart	services/ollama_client.dart, ollama_config.dart, pedagogy_client.dart, pedagogy_config.dart, ticket_client.dart, notifications_client.dart, feedback_client.dart	Variables de build (CAMPUS_API_BASE_URL, ...)

Client	Technologie	Fichiers d'intégration	Configuration de l'URL
Web	React	src/services/api.js — classe CampusCompanionAPI unique	Variable d'environnement REACT_APP_API_BASE_URL
Desktop	Electron	renderer/app.js — constante API_BASE_URL + helpers apiFetchJson/apiPatchJson	Constante en dur (adaptable en configuration)

Les trois équipes ont fait des choix d'implémentation différents (classes typées par domaine en Dart, classe unique de façade en JS, fonctions utilitaires en Electron) sans jamais avoir eu besoin de modifier l'API : la même base d'endpoints, les mêmes formats JSON et le même mécanisme d'authentification ont suffi aux trois.

3. Intégration mobile (Flutter / Dart)

Le client mobile encapsule chaque activité métier de l'API dans une classe dédiée (OllamaClient pour le chat, TicketClient pour le fallback, PedagogyClient, NotificationsClient...), configurée via des variables d'environnement de build plutôt que codée en dur — ce qui permet de pointer le même client vers un environnement de développement, de test ou de production sans recompiler la logique métier.

3.1 Configuration (ollama_config.dart)

```
// mobile/lib/services/ollama_config.dart
class OllamaConfig {
  final String baseUrl;

  factory OllamaConfig.fromEnv() {
    const campusBaseUrl = String.fromEnvironment('CAMPUS_API_BASE_URL', defaultValue: '');
    const pedagogyBaseUrl = String.fromEnvironment('PEDAGOGY_API_BASE_URL', defaultValue: '');
    const ollamaBaseUrl = String.fromEnvironment('OLLAMA_BASE_URL', defaultValue: '');
    final selected = campusBaseUrl.trim().isEmpty
      ? campusBaseUrl
      : (pedagogyBaseUrl.trim().isEmpty ? pedagogyBaseUrl : ollamaBaseUrl);
    return OllamaConfig(baseUrl: _normalizeBaseUrl(
      selected.trim().isEmpty ? 'http://localhost:8081' : selected));
  }

  Uri get chatUri => Uri.parse('${baseUrl}/api/chat');
}
```

3.2 Appel du chat (ollama_client.dart)

```
// mobile/lib/services/ollama_client.dart (extrait)
class OllamaClient {
  final OllamaConfig config;

  Future<OllamaChatResponse> chat({
    required List<OllamaChatMessage> messages,
    String? sessionId,
    int? userId,
  }) async {
    final Uri url = config.chatUri;
    final body = <String, dynamic>{
      'question': _extractLastUserQuestion(messages),
      'sessionId': sessionId,
      'userId': userId,
    }..removeWhere((_, v) => v == null);

    final res = await http.post(url,
```

```

        headers: const {'Content-Type': 'application/json'},
        body: jsonEncode(body));

    if (res.statusCode < 200 || res.statusCode >= 300) {
      throw OllamaException(message: 'Ollama chat request failed',
        uri: url, statusCode: res.statusCode);
    }
    return OllamaChatResponse.fromJson(jsonDecode(res.body));
  }
}

```

3.3 Fallback ticket (ticket_client.dart)

```

// mobile/lib/services/ticket_client.dart (extrait)
class TicketClient {
  Future<int> sendFallback({
    required String question,
    required String studentName,
    required String studentFirstname,
    required String studentClass,
    required String studentEmail,
  }) async {
    final url = Uri.parse('${config.baseUrl}/api/tickets/fallback');
    final response = await http.post(url,
      headers: const {'Content-Type': 'application/json'},
      body: jsonEncode({
        'question': question, 'studentName': studentName,
        'studentFirstname': studentFirstname,
        'studentClass': studentClass, 'studentEmail': studentEmail,
      }));
    if (response.statusCode == 201 || response.statusCode == 200) {
      return jsonDecode(response.body)['id'] as int? ?? 0;
    }
    throw TicketException(message: 'Impossible de créer le ticket',
      statusCode: response.statusCode);
  }
}

```

4. Intégration web (React)

Le client web centralise l'intégralité de l'intégration dans une seule classe, `CampusCompanionAPI`, instanciée une fois et réutilisée dans tous les composants. Cette approche illustre directement l'exigence de généricité : un seul point d'entrée, une méthode générique `_fetch()` avec gestion du timeout et des erreurs HTTP, et une méthode dédiée par service métier de l'API (chat, tickets, documents, knowledge, interactions, users).

4.1 Extrait — src/services/api.js

```

// web/src/services/api.js (extrait)
class CampusCompanionAPI {
  constructor(baseUrl = null) {
    this.baseUrl = baseUrl || process.env.REACT_APP_API_BASE_URL || 'http://localhost:8080';
    this.timeout = parseInt(process.env.REACT_APP_API_TIMEOUT || '30000', 10);
  }

  async _fetch(endpoint, options = {}) {
    const url = `${this.baseUrl}${endpoint}`;
    const controller = new AbortController();
    const timeoutId = setTimeout(() => controller.abort(), this.timeout);
    try {
      const response = await fetch(url, {
        headers: { 'Content-Type': 'application/json' },

```

```

        ...options,
        body: options.body ? JSON.stringify(options.body) : undefined,
        signal: controller.signal,
    });
    clearTimeout(timeoutId);
    if (!response.ok) {
        const errorData = await response.json().catch(() => ({}));
        const error = new Error(errorData.error || `HTTP ${response.status}`);
        error.status = response.status;
        throw error;
    }
    return response.json();
} catch (error) {
    clearTimeout(timeoutId);
    if (error.name === 'AbortError') throw new Error(`Timeout après ${this.timeout}ms`);
    throw error;
}
}

async chat(question, userId = null, sessionId = 'web-session') {
    return this._fetch('/api/chat', { method: 'POST', body: { question, userId, sessionId } });
}

async submitTicketFallback(studentName, studentFirstname, studentClass, studentEmail, question) {
    return this._fetch('/api/tickets/fallback', {
        method: 'POST',
        body: { studentName, studentFirstname, studentClass, studentEmail, question },
    });
}

async sendFeedback(interactionId, feedback) {
    if (!['UP', 'DOWN'].includes(feedback)) throw new Error('Feedback doit être "UP" ou "DOWN"');
    return this._fetch(`/api/interactions/${interactionId}/feedback`, { method: 'PATCH', body: {
feedback } });
}
}

export default CampusCompanionAPI;

```

Le commentaire d'usage inclus dans le fichier source (« `const api = new CampusCompanionAPI(); const response = await api.chat(...)` ») illustre explicitement l'intention de réutilisabilité recherchée par l'équipe.

5. Intégration desktop (Electron)

Le client desktop adopte une approche plus légère à base de fonctions utilitaires (`apiFetchJson`, `apiPatchJson`) plutôt qu'une classe : l'objectif métier reste identique — centraliser l'ajout du header `Authorization` et la gestion des erreurs HTTP en un seul endroit, pour que le reste de l'application n'ait jamais à connaître les détails d'authentification.

5.1 Extrait — `renderer/app.js`

```

// desktop/renderer/app.js (extrait)
const API_BASE_URL = "http://localhost:8080";

const Auth = (() => {
    async function login(email, password) {
        const res = await fetch(API_BASE_URL + "/api/auth/login", {
            method: "POST",
            headers: { "Content-Type": "application/json" },
            body: JSON.stringify({ email, password }),
        });
        if (!res.ok) return { ok: false, message: "Erreur de connexion." };
        const data = await res.json();
    }
}

```

```

    set({ token: data.token, user: { id: data.userId, email: data.email, role: data.role } });
    return { ok: true };
  }
  return { login, get, set, clear, isAuthenticated };
})();

async function apiFetchJson(path, options = {}) {
  const token = Auth.get()?.token;
  const headers = { ...(options.headers || {}) };
  if (token && !headers.Authorization) headers.Authorization = `Bearer ${token}`;

  const res = await fetch(API_BASE_URL + path, { ...options, headers });
  if (!res.ok) {
    if (res.status === 401) { Auth.clear(); location.hash = "#/login"; render(); }
    throw new Error(`HTTP ${res.status}`);
  }
  return await res.json();
}

```

On notera la gestion homogène de l'expiration de session : une réponse 401 déclenche systématiquement l'effacement du jeton et la redirection vers l'écran de connexion, un comportement que l'API impose implicitement à tout client via un code de statut standard, sans documentation supplémentaire nécessaire.

6. Ce que les trois intégrations ont en commun

Au-delà des choix de langage et de style propres à chaque équipe, les trois clients reposent sur exactement le même contrat, ce qui constitue la preuve concrète du caractère générique et réutilisable de l'architecture définie pour l'API Campus Companion :

Élément du contrat	Identique sur les 3 clients ?	Détail
Base URL configurable	Oui	Variable d'environnement ou constante, jamais imposée par l'API
Format des requêtes/réponses	Oui	JSON strict, mêmes noms de champs (question, sessionId, ticketCreated...)
Authentification	Oui	Header Authorization: Bearer <token>, ajouté automatiquement par chaque client
Gestion des erreurs HTTP	Oui (logique équivalente)	Chaque client détecte les codes 4xx/5xx et adapte son affichage, sans cas particulier par client
Connaissance du code backend	Non requise	Aucun des trois clients n'a eu besoin de lire le code source Spring Boot pour s'intégrer